

Includere libreria	Library nome_libreria;
Includere package	Use nome_libreria.nome_package.nomeoggetto;
Dichiarazione di entità	Entity nome_entità is generic (nome_costante : tipo ; nome_costante : tipo ; nome_costante : tipo); port (nome_segnaled : modo tipo_segnaled ; nome_segnaled : modo tipo_segnaled); end nome_entità;
Dichiarazione di segnali	signal nome_segnaled : tipo_segnaled ;
Dichiarazione di costanti	constant nome_costante : tipo_costante ;
Dichiarazione di componenti	component nome_componente is port (nome_segnaled : modo tipo_segnaled ; nome_segnaled : modo tipo_segnaled ; nome_segnaled : modo tipo_segnaled); end component ;
Istanza componente*	nome_label : nome_componente port map (segnale1, segnale2, ..., segnaleN);
Istanza ripetitiva*	nome_label : for identificatore in range generate istanza_componente; end generate ;
Assegnazione selezionata*	with segnale_selezione select segnale <= valore_a when caso_a, valore_b when caso_b, valore_n when caso_n;
Assegnazione condizionata*	segnale <= valore_a when condizione1 else valore_b when condizione2 else valore_n;
Dichiarazione di variabili**	Variable nome_variable : tipo ;

Statements if....then

```
if espressione_booleana  
then  
statements_sequenziali;  
endif;
```

```
if espressione_booleana  
then  
statements_sequenziali;  
else  
statements_sequenziali;  
endif;
```

```
if espressione_booleana  
then  
statements_sequenziali;  
elseif espressione_booleana  
then  
statements_sequenziali;  
elseif espressione_booleana  
then  
statements_sequenziali;  
....  
endif;
```

```
if espressione_booleana  
then  
statements_sequenziali;  
elseif espressione_booleana  
then  
statements_sequenziali;  
elseif espressione_booleana  
then  
statements_sequenziali;  
....  
else  
statements_sequenziali;  
endif;
```

Statement case...when

```
case segnale_o_variabile_di_selezione is  
when caso_a =>  
statements_sequenziali;  
.....  
when caso_a =>  
statements_sequenziali;  
end case;
```

Statement for...loop

```
for identificatore in range loop  
statements_sequenziali;  
end loop;
```

Statement while....loop

```
while espressione_booleana loop  
    statements_sequenziali;  
end loop;
```

Definizione di tipi enumerati

```
type nome_tipo is (elemento, elemento, elemento);
```

Definizione di array

```
type nome_tipo is array ( start to end) of tipo_elemento;  
type nome_tipo is array ( start downto end)  " "  ;  
type nome_tipo is array (tipo_indice) of tipo_elemento;
```

```
type nome_tipo is array  
    (tipo_indice range start to end) of tipo_elemento;
```

```
type nome_tipo is array  
    (tipo_indice range start downto end)  
    of tipo_elemento;
```

Definizione di sottotipi

```
subtype nome_sottotipo is nome_tipo_scalare start to end;  
subtype nome_sottotipo is nome_tipo_scalare  
    start downto end;
```

```
subtype nome_sottotipo is nome_tipo_array_unconstrained  
    (start to end);
```

```
subtype nome_sottotipo is nome_tipo_array_unconstrained  
    (start downto end);
```

Definizione di funzioni

```
function nome_funzione (  
    nome_parametro1 : in tipo_parametro;  
    .....  
    nome_parametroN : in tipo_parametro)  
return tipo_risultato is
```

```
dichiarazioni di tipo;  
dichiarazione di variabili;**  
dichiarazione di costanti;  
dichiarazione di funzioni;
```

```
begin  
statements_sequenziali;  
.....  
statements_sequenziali;ù  
return valore;  
end nome_funzione;
```

Definizione di procedure

```
procedure nome_procedura (  
nome_parametro1 : modo tipo_parametro;  
.....  
nome_parametroN : modo tipo_parametro) is
```

```
dichiarazioni di tipo;  
dichiarazione di variabili;**  
dichiarazione di costanti;  
dichiarazione di funzioni;
```

```
begin  
statements_sequenziali;  
.....  
statements_sequenziali;  
end nome_procedura;
```

Definizione package

```
package nome_package is
```

```
dichiarazioni_di_tipo  
dichiarazioni_di_sottotipo  
dichiarazioni_di_costanti  
dichiarazioni_di_segnali  
dichiarazioni_di_funzioni  
dichiarazioni_di_procedure  
dichiarazioni_di_componenti
```

```
end nome_package;
```

```
package body nome_package is
```

```
definizioni_di_funzioni  
definizioni_di_procedure
```

```
end nome_package;
```

Dichiarazione architettura

```
Architecture nome_architettura of nome_entità is
```

```
dichiarazioni_di_tipo  
dichiarazioni_di_sottotipo  
dichiarazioni_di_costanti  
dichiarazioni_di_segnali  
dichiarazioni_di_funzioni  
dichiarazioni_di_procedure  
dichiarazioni_di_componenti
```

```
begin  
statements_concorrente  
...  
statements_concorrente  
end nome_architettura;
```

Processo	<pre> label : process(segnale_a, segnale_b, ..., segnale_n); dichiarazioni di tipo; dichiarazione di variabili;** dichiarazione di costanti; begin statement sequenziale; statement sequenziale; statement sequenziale; end process; </pre>
Statement di aggregazione	<pre> nome_segnaled_vettore <= (nome_segnaled1, ..., nome_segnaledN); </pre>
Fronte di salita/discesa del clock (è una condizione) **IEEE 1164 salita **IEEE 1164 discesa	<pre> (clk='1' / '0' and clk'event) (clk='1' / '0' and not clk'stable) Rising_edge(clk) Falling_edge(clk) </pre>
Array unconstrained	<pre> Type std_logic_vector is array (natural range <>) of std_logic; </pre>